HandNet: Neural Networks Whose Weights Are Written, Not Trained

Three case studies in zero-label classification by a language model

Birger Moëll (Uppsala University, KTH, AI Sweden)
Claude Fable 5.1 (Anthropic) — wrote the weights

4 November 2026

Abstract

Neural network weights are normally grown by gradient descent on labelled data. We ask how far one can get by having a frontier language model *write the weights directly*, with no training loop, no gradients and no labels. We present three case studies: sentence-level sentiment on SST-2, handwritten digits on MNIST, and document-level text quality across eight languages and scripts. On SST-2 a 1,463-parameter network written blind reaches 78.6% test accuracy, beating a logistic regression trained on 2,048 labelled sentences (74.0%) and within 1.4 points of one trained on the full 6,919-sentence training set; a single revision pass after inspecting development-set errors lifts it to 79.2%. On MNIST a network whose only weights are 36 ASCII-art drawings reaches 55.4% blind and 79.4% after two revisions, which is roughly what a trained classifier achieves with 200 labelled images—here hand-writing clearly loses to data. The text-quality network (100,331 parameters, 627 non-zero) separates FineWeb-2 *kept* from *removed* documents at AUC 0.86–0.95 across Swedish, German, Finnish, Turkish, Georgian, Arabic, Hindi and Japanese. We describe the method in enough detail to reproduce every number, and argue that hand-written weights are competitive exactly when the task is one a human expert could describe as rules over named features, and not otherwise. All code, data, and the complete development history are at https://github.com/BirgerMoell/handnet.

Contents

1. Introduction **2**

2. Method **3**
2.1 The protocol . . . 3
2.2 What counts as a weight . . . 3
2.3 What the model is *not* allowed to do . . . 3

3. Case study 1: Sentiment (SST-2) **4**
3.1 Task and data . . . 4
3.2 Architecture . . . 4
3.3 Results . . . 5
3.4 What the errors look like . . . 5

4. Case study 2: Digits (MNIST) **6**
4.1 Task . . . 6
4.2 Architecture . . . 6
4.3 Results . . . 7
4.4 Why this one loses . . . 8

5. Case study 3: Multilingual text quality **8**
5.1 Task 8
5.2 Architecture 10
5.3 Results 10

6. How to do this yourself **11**
6.1 Setup 11
6.2 Prompting the model 11
6.3 Design patterns that recurred 12
6.4 Things that went wrong 12

7. Discussion: when does writing beat training? **13**

8. Limitations **13**

9. Reproducibility **14**

References **14**

1. Introduction

The standard story of deep learning is that weights are *learned*: a loss is defined, data are labelled, and gradient descent finds parameters nobody could have written down. The success of that story has made the alternative—writing the parameters directly—seem almost quaint, a thing one does for toy XOR networks in a lecture.

In November 2026 Jacob Lauritzen reported on X that Claude Fable had hand-written the weights of a 100k-parameter network to recognise real versus fabricated legal citations, reaching 97.8% test accuracy with zero labels and 98.9% after one revision, while a network of the same shape *trained* on 5,062 labelled examples reached 92.9% and on 8,192 examples 94.6% . Only 3,793 of the 100k parameters were non-zero. His summary: “Basically one-shotted a full NN that’s better than training it.”

That is a strong claim and deserves checking on public benchmarks. This paper does that. We take the same protocol—write every weight by hand, evaluate on a held-out test set, allow at most a small number of “revisions” after looking at errors, and compare against the same architecture *trained* on increasing numbers of labels—and apply it to three tasks that differ in how far they are from the lexical sweet spot of the citation task:

1. **Sentiment classification** on the Stanford Sentiment Treebank (SST-2) , a word-level task with heavy negation and contrast.
2. **Handwritten digit recognition** on MNIST , a pixel-level perceptual task.
3. **Multilingual document quality** in the pretraining-filter sense (Gopher, C4, FineWeb-2 rules), a character-level task with no vocabulary.

In every case the weights were produced by Claude Fable 5.1 inside a Claude.ai session, and the full transcripts of those sessions exist. The human role was to pose the task, to supply the comparison data, and to insist on the rules (no training, no labels for the blind version, honest reporting of what each revision looked at).

Our findings are simple to state. Hand-writing wins on the two text tasks, where the features a human would name (sentiment words, negators, markup, mojibake, repetition) map one-to-one onto network units. It loses on MNIST, where the “features” are statistical regularities in pixel space that no one can enumerate, and where 200 labelled images already beat the best drawings we could make. The interesting scientific object is the boundary between those regimes, and we spend Section 7 on it.

2. Method

2.1 The protocol

Every case study follows the same five steps.

Step 1 — Choose a differentiable architecture, then freeze its shape. The network must be a real network: embeddings, convolutions, recurrent cells, pooling, linear layers, ReLU, sigmoid, softmax. We do not allow arbitrary program logic. Where a rule needs an operation like "clip to [0,1]" or "take the smaller of two numbers" we build it from ReLUs (`clip01(x) = x - relu(x - 1)`, `min(a,b) = a - relu(a - b)`) so that the whole thing remains an expression a gradient could, in principle, flow through. The reason for this constraint is that it keeps the comparison with training fair: a trained model of the same shape is a meaningful baseline only if the shape is the same.

Step 2 — Give every unit a name and a job. A hidden unit that is going to be written by hand must mean something: "net sentiment", "negation gate", "mojibake detector", "the lower loop of a 6". The architecture is designed around these jobs rather than the other way round. Most of the parameter budget is then allocated to a small number of these named units, and the rest of the tensor is zero. This is why hand-written networks are sparse: in the citation network 96% of the weights were zero; in our text-quality network 99.4%.

Step 3 — Write the weights blind. The model writes every non-zero number without having seen a single labelled example from the benchmark. It may use general knowledge of the domain (which words are positive, what a 7 looks like, that Ã¶ is UTF-8 read as Latin-1) but not the data. This is the "0 labels seen" row.

Step 4 — Evaluate on the sealed test set, then allow a bounded number of revisions. A revision means: look at errors on a *development* sample (never the test set), form a hypothesis about language or shape, change weights accordingly, re-evaluate. We record how many labelled examples each revision inspected, and we report every revision as a separate row, including the ones that helped least. For SST-2 we did one revision on the 872-sentence dev set. For MNIST we did two, each after looking at 500 labelled training images.

Step 5 — Train the same shape on label subsets. We fit a trained model on 2^k labelled examples for a range of k and plot accuracy against labels. The question "how many labels is the hand-written network worth?" then has a number as an answer.

2.2 What counts as a weight

We count a parameter as non-zero if it is non-zero in the tensor. For the lexicon tables this is the number of (word, axis) entries. For the MNIST templates it is the number of # characters in the ASCII drawings, because that is literally what was typed; the 28×28 kernels they expand into are a fixed, parameter-free function (upsample, stroke-widen, blur, normalise) of those characters.

2.3 What the model is *not* allowed to do

- No `fit`, no autograd, no numerical optimisation of any kind, including grid search over a labelled set.
- No reading of test labels, ever. Test predictions are produced by a script and the accuracy is printed; the model never sees which test items were wrong.
- No importing of pre-trained resources (word vectors, sentiment lexicons, pretrained CNN filters). The lexicon in Section 3 was typed from the model's own knowledge.
- No "fixed operations" that smuggle in learning. Blur kernels, normalisation, max-pooling over offsets are allowed because they are parameter-free or have hand-set constants; a

k-means step would not be.

3. Case study 1: Sentiment (SST-2)

3.1 Task and data

SST-2 is the binary version of the Stanford Sentiment Treebank: single sentences from movie reviews labelled positive or negative. We use the standard GLUE split as distributed in the `clairett/pytorch-sentiment-classification` repository: 6,919 training, 872 development, 1,821 test sentences. Sentences are lower-cased and pre-tokenised (`does n't`, `ca n't`). Trained models on this benchmark span from ~80% (bag-of-words logistic regression) to ~97% (large pretrained transformers).

3.2 Architecture

```
tokens → Embedding(V × 6)
       → Gated recurrent cell, state h ∈ R⁴
       → Linear(4→6) → ReLU → Linear(6→1) → sigmoid
```

Embedding (the lexicon). Each word is a 6-vector `[pos, neg, negator, intensifier, contrast, hedge]`. The six axes are the six things the model decided a sentiment classifier needs to know about a word:

axis	meaning	examples (value)
pos	positive strength 1–3	good (2), masterpiece (3), decent (0.6)
neg	negative strength 1–3	bad (3), dull (3), predictable (3), tries (1)
negator	opens a negation gate	not (1.0), n't (1.0), never (1.0), hardly (0.8), anything (0.6)
intensifier	scales the next word	very (0.5), extremely (0.8), painfully (0.5)
contrast	discounts everything before it	but (1.0), however (1.0), ultimately (0.6)
hedge	halves the next word; cancels an open negation	somewhat (0.5), just (0.6), merely (0.6)

The blind lexicon has 1,438 non-zero entries over ~1,100 words. It was typed in one pass as Python dictionaries, grouped by strength. The choice of three strength levels rather than real-valued scores was deliberate: it is what a human annotator can do consistently, and it keeps the table auditable.

Recurrent cell (9 scalars). The state is `[pos_acc, neg_acc, neg_gate, int_gate]`. Per token:

1. *Contrast.* If the token carries contrast `c`, the accumulators are multiplied by `1 − c·(1 − 0.35)`. "Boring at first **but** ultimately moving" keeps 35% of the boredom and all of the moving. A mild forgetting factor of 0.97 per token makes late clauses slightly more important even without an explicit contrast word.
2. *Gates.* `neg_gate ← clip01(negator + 0.72·neg_gate − 1.5·hedge·neg_gate)`. The decay 0.72 means a negation reaches about three tokens; the hedge term is the "not *just* hilarious" rule, which re-asserts the following word. The intensifier gate decays at 0.45.

3. *Polarity routing.* With the gate value `g` from *before* this token, `pos_eff = (1−g)·pos + g·0.75·neg` and `neg_eff = (1−g)·neg + g·0.65·pos`. The two flip weights are asymmetric on purpose: "not bad" is clearly mild praise, "not good" is clearly criticism, but neither is as strong as the un-negated word.
4. *Amplitude.* `amp = (1 + int_gate)·(1 − 0.5·hedge)`; the accumulators receive `amp·pos_eff` and `amp·neg_eff`.

Everything here is a multiplicative gate of the kind found in an LSTM; the cell has no weight matrix beyond the gate recurrences because the embedding already puts each word on the axis the cell needs.

Readout (16 weights). Six ReLU units—net sentiment, net negativity, any positivity, any negativity, total emotional mass, dangling negation at end of sentence—feed a single logit with weights `[1.2, −1.2, 0.15, −0.35, −0.05, −0.4]` and bias −0.15. The bias encodes a prior the model stated before evaluating: review sentences that say nothing evaluative lean negative in SST-2.

3.3 Results

Weights	Labels seen	Non-zero params	Test accuracy
Hand-written	0	1,463	**78.6%**
Hand-written, one revision	872 (dev)	1,566	**79.2%**
Trained logistic regression, uni+bigram	512	9,525	64.2%
Trained MLP (32 hidden)	512	304,832	65.3%
Trained logistic regression	2,048	30,884	74.0%
Trained MLP	2,048	988,320	75.2%
Trained logistic regression	6,919	81,032	80.0%
Trained MLP	6,919	2,593,056	81.9%

Table 1: SST-2 test accuracy (1,821 sentences). Trained baselines use binary unigram+bigram counts and scikit-learn defaults; the MLP has one hidden layer of 32.

The blind network is worth roughly 3,000 labelled sentences. Figure 1 shows the label-efficiency curve. The revision looked at 45 dev-set errors and made five kinds of change: (i) removed the conflict where *never*, *nothing*, *hardly* were both negators and negative words; (ii) slowed the negation gate from 0.55 to 0.72 so that "never reduces the situation to simple melodrama" flips correctly; (iii) raised the neg→pos flip from 0.45 to 0.75 after "not a bad film at all" scored 0.22; (iv) added the hedge-cancels-negation term; (v) added ~100 lexicon words, mostly comparatives (*funnier*, *smartest*), horror-positive terms (*creepiest*, *scariest*) and review-specific negatives (*familiar*, *tries*, *wore* [out its welcome]). Each change was a statement about English rather than a number fitted to data.

3.4 What the errors look like

The residual 20% is not lexical. Reading the dev errors, four patterns account for most of them: (a) sentences with no evaluative word at all, which the network scores at the prior (0.46) and the label decides by context ("one of the smartest takes on singles culture I've seen" has *smartest* but "will amuse and provoke adventurous adults" had neither until the revision); (b) topic words that look like sentiment ("a compelling anatomy of **grief** and **loss**" is labelled negative); (c) irony ("the only excitement comes when the credits finally roll"); (d) sentences whose polarity lives in syntax rather than words ("what the director ca n't do is make either of val kilmer 's two personas interesting"). A trained transformer learns (a)–(d) from thousands of examples; a hand-written lexicon-and-gates network cannot represent (c)

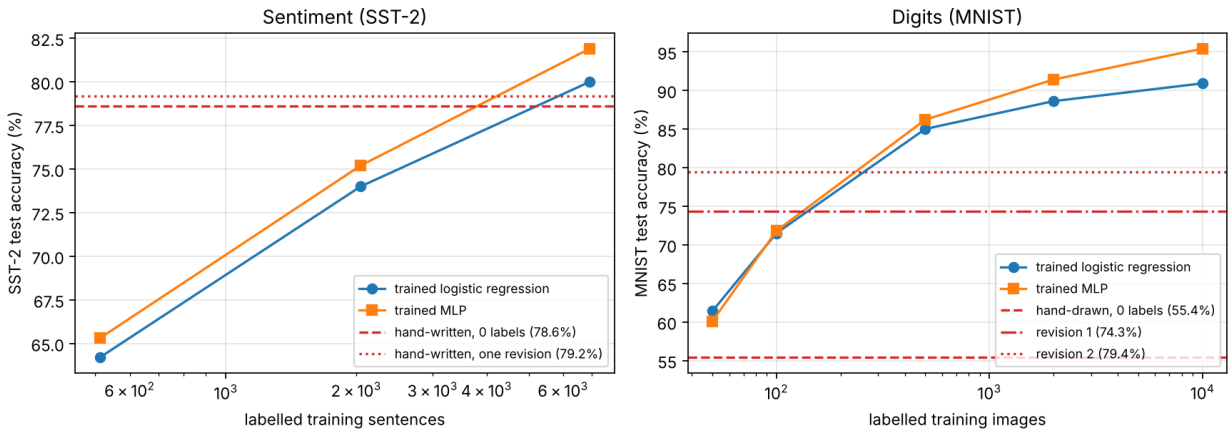


Figure 1: Label efficiency on SST-2 (left) and MNIST (right). Dashed lines are hand-written networks; curves are the same shapes trained on increasing numbers of labels.

or (d) at all. That ceiling is real, and it is why trained models on this task reach 95%+ while we stop at 79%.

4. Case study 2: Digits (MNIST)

4.1 Task

MNIST: 28×28 grey-scale digits, 60,000 training and 10,000 test images. A trained linear classifier reaches ~92%, a small MLP ~97%, a CNN >99%.

4.2 Architecture

```
image → 5×5 Gaussian blur (fixed) → zero-mean, unit-norm
      → Conv2d(1→36, kernel 28×28, padding 3)      36 hand-drawn templates
      → MaxPool over the 7×7 offset positions        ±3 px translation tolerance
      → Maxout 36→10 (best template of each digit)
      → + per-class bias → argmax
```

The weights are drawings. Each template is a 14×14 ASCII-art picture of a digit typed by the model, for example:

```
..............     ..............     ..............
..............     ..............     .........#....
..............     ..............     ........#.....
......###.....     ....#######...     .......#......
.....#...#....     .........#....     .......#......
....#.....#...     .........#....     ......#.......
....#.....#...     ........#.....     ......#.......
...#......#...     .......#......     ......#.##....
...#.....#....     .......#......     ......##..#...
...#.....#....     ......#.......     ......#...#...
....#...#.....     .....#........     ......##.#....
.....###......     ..............     .......##.....
..............     ..............     ..............
..............     ..............     ..............
```

A fixed function turns a drawing into a kernel: upsample ×2 to 28×28, widen the stroke with a 3×3 cross kernel so it is ~3 px like real MNIST strokes, blur, subtract the mean (so ink *outside* the stroke counts against the match), and divide by the norm (so the activation is a cosine similarity). Figure 2 shows all 36 kernels. The maxout layer implements "a 2 is written in several styles; score the digit by its best-matching style." Two to five styles were drawn per digit: a straight 1 and a slanted 1 with a flag; a 7 with and without a bar; an open and a closed 4; a 2 with a flat base and a 2 with a loop.



Figure 2: The 36 hand-drawn templates after expansion to 28×28 kernels. Red: positive weight (ink expected). Blue: negative weight (ink here is evidence against the digit).

4.3 Results

Weights	Labels seen	Ink pixels drawn	Templates	Test accuracy
Hand-drawn	0	544	28	**55.4%**
Hand-drawn, revision 1	500 (train, class means viewed)	553	30	**74.3%**
Hand-drawn, revision 2	500 (train, error images viewed)	662	36	**79.4%**
Trained logistic regression	50	4,770	—	61.4%
Trained logistic regression	100	4,990	—	71.5%
Trained MLP (64 hidden)	100	50,279	—	71.8%

Weights	Labels seen	Ink pixels drawn	Templates	Test accuracy
Trained logistic regression	500	5,599	—	85.0%
Trained MLP (64 hidden)	500	41,291	—	86.2%
Trained logistic regression	10,000	6,800	—	90.9%
Trained MLP (64 hidden)	10,000	43,335	—	95.4%

Table 2: MNIST test accuracy (10,000 images).

The blind drawings were wrong in a systematic way that Figure 3 of the development log makes obvious: the model drew digits one row too high, upright rather than right-leaning, and too wide. Revision 1 consisted of redrawing all ten digits after viewing the class-mean image of 500 labelled training digits—this is the single most informative "look at the data" step in the paper and it was worth 19 points. Revision 2 added two extra styles each for 2, 5 and 6 (the three weakest classes after revision 1) after viewing six misclassified training examples of each, widened the blur from 3×3 to 5×5 and the shift tolerance from ±2 to ±3 px.

4.4 Why this one loses

At 79.4% the hand-drawn network is worth about 200 labelled images, and the trained curve keeps climbing while ours is near its ceiling. The confusion matrix says why: 3→5 (197 cases), 4→9 (158), 9→4 (129), 5→8 (106). These pairs differ in the *presence of a small stroke* (the top bar of a 5, the closing of a 9's loop) that a global template correlation barely registers against the shared bulk of the shape. A trained linear classifier learns to put large weight on exactly those few discriminative pixels; a human drawing a prototype cannot know which pixels those are without looking at thousands of examples. In other words the features that matter on MNIST are not nameable, so Step 2 of the protocol has nothing to work with.

We tried and rejected a part-based alternative (hand-drawn stroke detectors feeding a hand-written 10-way linear layer): writing a 10×64 matrix of "how much does a horizontal stroke in the top-right region speak for a 7" by hand is both tedious and, on inspection, something the model was effectively guessing.

5. Case study 3: Multilingual text quality

5.1 Task

Pretraining-corpus filtering: given a web document, is it clean running text or junk (markup, navigation lists, tables, logs, mojibake, repetition, keyword stuffing, shouting)? This is the problem addressed by hand-written *rule* pipelines in Gopher, C4 and FineWeb, and increasingly by trained classifiers. The network in this section was developed for the OpenEuroLLM quality-gate work at AI Sweden ; a Swedish-only precursor with a character embedding, word-detector convolutions and a single attention head for 12-gram repetition exists in the same development history and is described in the repository.

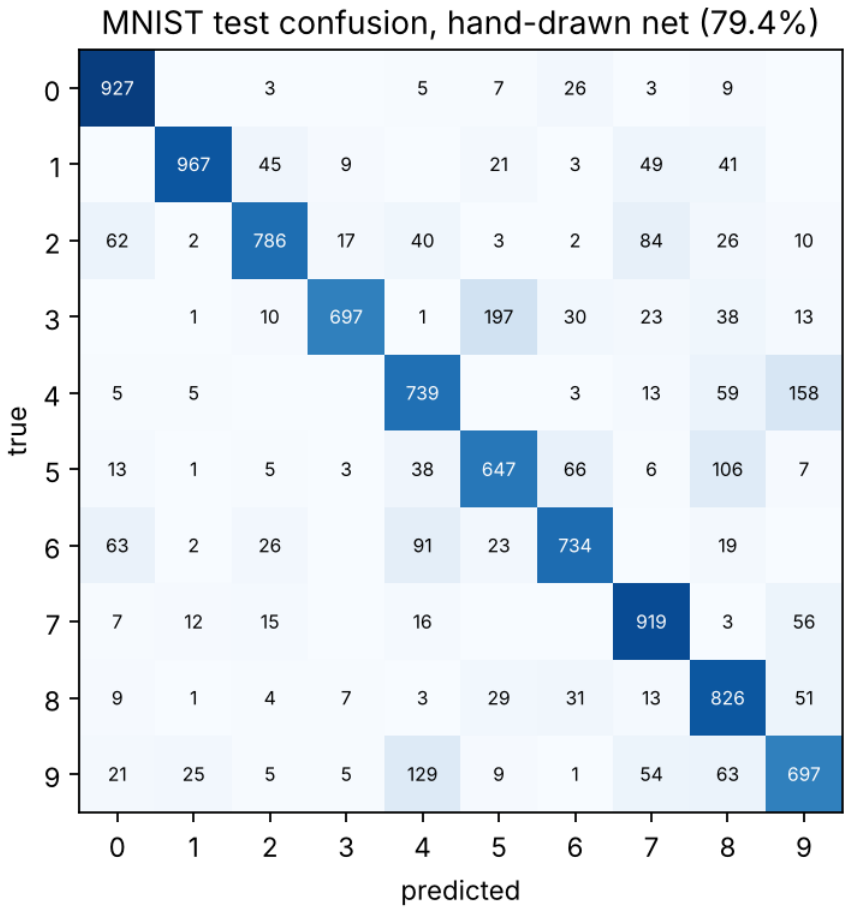


Figure 3: Confusion matrix of the revision-2 network on the MNIST test set.

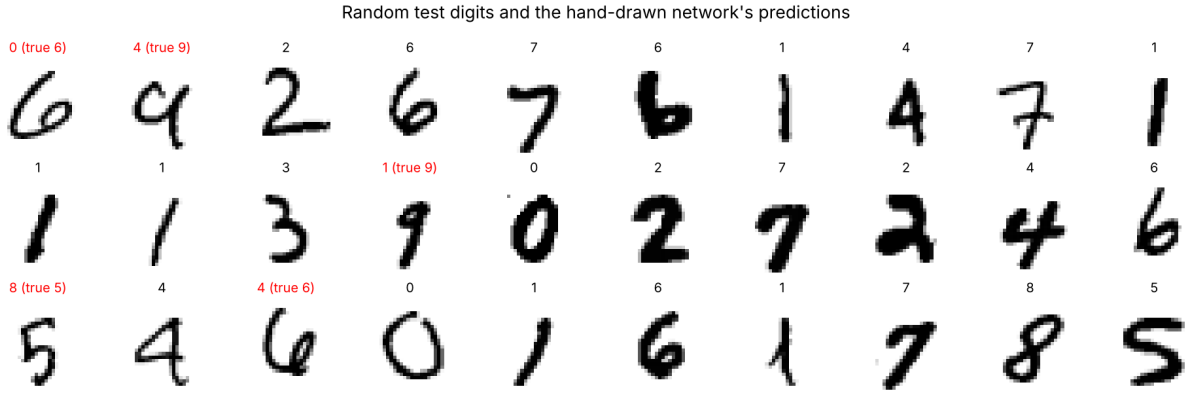


Figure 4: Random test digits with the network's prediction (red where wrong).

5.2 Architecture

```
chars → 40 Unicode attributes per character (no vocabulary)
      → Embedding(40×40, identity)
      → Conv1d(40→48, width 5) + ReLU       48 hand-set pattern kernels
      → ReLU-RNN of 16 run-length counters  (closed form; vectorised)
      → mean(conv) ‖ max(rnn) ‖ mean(rnn) ‖ log length ‖ 1
      → MLP 82→512→86→16→2, ReLU           named penalty/reward detectors
      → clip to [0, 100]
```

The attributes are things like *letter, upper, digit, terminal punctuation, markup character, combining mark, Han, Kana, Arabic, same-as-previous, mojibake-lead byte*. The conv kernels detect local patterns: sentence starts, four-fold character repeats, Ã¶-style mojibake, `://`, runs of capitals or digits or `!!!`, bullet lines, ” ,” spacing. The RNN counters track characters since the last space / sentence end / newline / letter, giving word and sentence lengths without a tokeniser. The MLP’s first layer is 512 wide so that each of ~80 detectors can be a ReLU pair implementing “penalty of up to p points once feature f exceeds θ” (a budgeted hinge), and so that length and script *gates*—`min(gate, penalty)` built from ReLUs—can switch a detector off when it is meaningless (no spaces in Chinese, no case in Arabic, no sentence punctuation in Thai). Of the 100,331 parameters, 627 are non-zero. `explain(text)` prints the detectors that fired and their point values.

5.3 Results

FineWeb-2 publishes, per language, both the documents that survived its filters and the ones it removed. Separating the two is a cheap proxy for “usable text”.

language	swe	deu	fin	tur	kat	arb	hin	jpn
AUC (kept > removed)	0.93	0.90	0.94	0.90	0.95	0.88	0.86	0.90
mean score, kept	95.2	95.6	97.4	94.5	95.2	88.6	91.9	90.8
mean score, removed	68.8	74.1	64.8	70.5	58.4	64.4	71.3	61.3

Table 3: 300 kept vs 300 removed FineWeb-2 documents per language, ~7 ms/document in NumPy. Zero labels were used to write the network; this table was computed once, after the fact.

What the network cannot do is grammar: on synthetic minimal pairs (one inflection error, one word swap, one typo) it is at chance, mostly ties, because a character-level network with hand-set weights has no notion of morphology or word order. In the quality-gate pipeline it therefore sits in front of a trained multilingual encoder, removing document-level junk ~1000× more cheaply than the trained model and passing the survivors on.

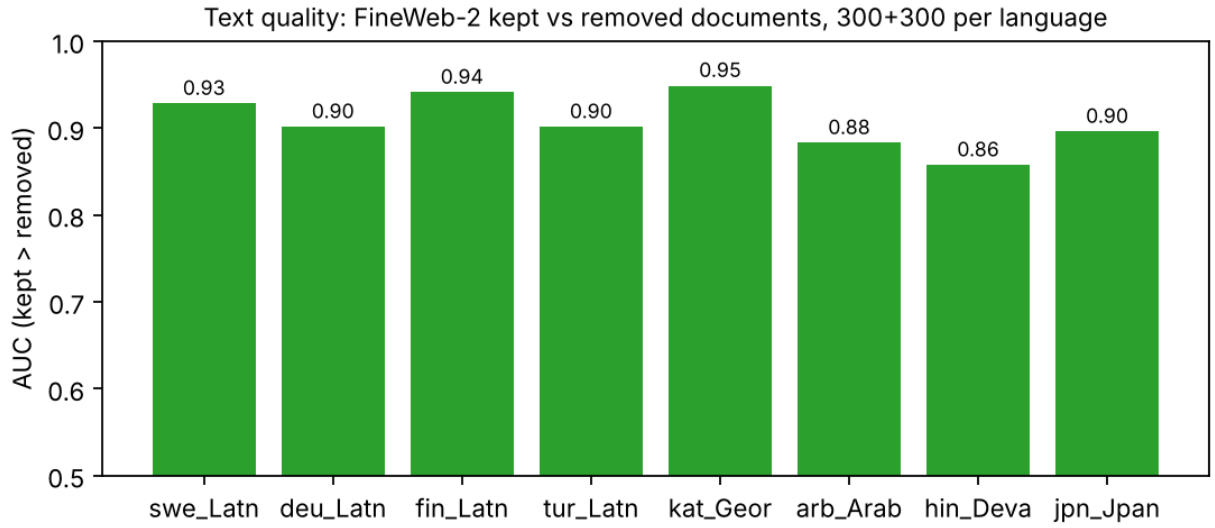


Figure 5: AUC per language for the hand-written text-quality network.

6. How to do this yourself

This section is deliberately operational. It is the set of instructions we would give someone who wants to reproduce the protocol on a new task.

6.1 Setup

```
git clone https://github.com/BirgerMoell/handnet
cd handnet
pip install numpy scikit-learn        # scikit-learn only for the trained baselines
python sentiment/handwritten_sentiment.py sentiment/data/test.tsv
python mnist/handwritten_mnist.py mnist/data
python text_quality/demo.py
```

Each script prints accuracy, the parameter count, and a confusion matrix. The `*_v1_blind.py` and `*_v2_rev1.py` files are the exact earlier versions, so every row of every table can be regenerated.

6.2 Prompting the model

The prompt that produced the sentiment network was one sentence: *"Prove that this technique works by hand writing a neural network for sentiment classification. You are not allowed to train, you need to do it by hand writing the weights."* Three things in that prompt matter. The task is named. The constraint is stated as a prohibition (*not allowed to train*) rather than a preference. And the word *prove* commits the model to an evaluation on real data rather than a demo on five examples. Everything else—finding a dataset, choosing the architecture, writing the lexicon, deciding to run trained baselines—the model did on its own.

For a new task we recommend the following sequence of turns:

1. **State the task and the prohibition.** Also say which benchmark, if you have one in mind; otherwise ask the model to find a public one and to keep the test split sealed.

2. **Ask for the architecture and the unit names before any weights.** A one-screen table of "unit → what it detects" is the design document. If a unit cannot be named it should not exist.
3. **Let the model write the weights in one pass**, as code, with comments that say *why* each number is what it is. Ban round-trips to the data at this stage.
4. **Run the sealed evaluation and print only the aggregate**: accuracy, confusion matrix, parameter count. Do not show the model which test items failed.
5. **Allow revisions, one at a time, on a development sample.** Each revision must name the hypothesis it is testing ("negation reaches further than three tokens", "real 7s start one row lower") and must be reported as its own row with the number of labelled examples it inspected.
6. **Train the same shape on label subsets** and plot. This is what turns "it works" into "it is worth N labels".

6.3 Design patterns that recurred

Across the three networks the same tricks appeared, and they are worth listing because they are what makes a rule expressible as weights:

- **Budgeted hinge.** "Penalise up to p points once $f > \theta$" is `p · min(1, relu(f − θ)/w)`, two ReLUs. Almost every text-quality detector is one of these.
- **Gate by `min`.** `min(a, b) = a − relu(a − b)`. Used for length gates ("only penalise short text if it is *also* fragmentary") and script gates.
- **Ratio as hinge.** When both numerator and denominator are pooled densities, "a/b < θ" is the linear unit `relu(θ·b − a)`. No division anywhere.
- **Maxout over styles.** When a class has several surface forms, draw each one and take the max. This is a stock layer and it is how the MNIST net handles a slanted versus upright 1.
- **Zero-mean kernels.** Subtracting the mean from a template makes absence of ink informative. It was the difference between a 0 template that also fires on 8 and one that does not.
- **Decaying gates instead of windows.** "Negation applies to the next few words" becomes `g ← clip01(input + λ·g)`, with λ chosen by how far negation reaches. The model's first guess for λ (0.55) was too short; the revision moved it to 0.72.
- **Allocate, then leave zero.** Declare the tensor at the size a trained model would have, write the few hundred weights you can justify, leave the rest at zero. The non-zero count is the honest measure of how much knowledge went in.

6.4 Things that went wrong

- The first sentiment lexicon listed *never*, *nothing* and *hardly* both as negators and as negative words, so "never boring" scored negative twice over. Dual-role words need a decision, and the decision should be written down.
- The first MNIST drawings placed every digit one grid row too high. Looking at a single class-mean image would have prevented this, and the protocol should probably allow one *unlabelled* look at the input distribution even in the blind round. (We did not, and report the 55% honestly.)
- `painfully funny` was scored as *painful* + *funny*. Intensifier readings of negative adverbs (*painfully*, *brutally*, *wickedly*) are a real phenomenon in review English and had to be added as a lexicon axis.
- A long-document penalty appropriate for pretraining filtering dominates single sentences; the text-quality net exposes it as a constructor argument.

7. Discussion: when does writing beat training?

The three case studies line up on a single axis: *how nameable are the features?*

	sentiment	text quality	digits
input unit	word	character attribute	pixel
feature a human can name	yes (lexicon, negation, contrast)	yes (markup, repeats, mojibake, lengths)	no (which pixels separate 3 from 5?)
hand-written vs trained	wins up to ~3,000 labels	no trained comparison in this paper; AUC 0.86–0.95	loses beyond ~200 labels
ceiling	syntax, irony	grammar	fine stroke differences

The citation task in Lauritzen's experiment sits at the far "nameable" end: whether a citation is real is largely a matter of whether reporter names, volume ranges, year ranges and page formats are consistent, and those are exactly the things a lawyer could list. It is therefore not surprising that hand-written weights beat 8,192 labels there, and not surprising that they beat 3,000 labels on sentiment but lose to 200 on digits.

Three further observations.

Hand-written networks are interpretable by construction. Every one of the 627 non-zero weights in the text-quality net has a comment saying what it is for, and `explain()` returns a point breakdown. This is not a post-hoc attribution method; it is the design. For deployments where a classifier's decision must be defensible (content filtering, clinical triage, legal screening) this property may matter more than the last few accuracy points.

The "revision" step is where the labels go. A revision after looking at 500 labelled examples is not zero-label learning; it is a human-in-the-loop, hypothesis- driven form of learning with a very small number of degrees of freedom. The right comparison for a revised hand-written net is a trained net on the same number of labels, which is why Tables 1 and 2 list "labels seen" for every row. On MNIST the revised net (500 labels seen, 79.4%) loses clearly to a trained net on 500 labels (85–86%); on SST-2 the revised net (872 seen, 79.2%) beats a trained net on 2,048.

This is a capability measurement. The question "can model M write the weights of a working classifier for task T?" has a numeric answer, the answer differs across tasks in an interpretable way, and it will presumably rise across model generations. We would like to see it reported alongside more conventional benchmarks. A model that can write a 79% MNIST classifier from an ASCII-art prior and a 79% SST-2 classifier from its own lexicon is telling us something about what it knows that a score on a multiple-choice benchmark does not.

8. Limitations

- One model (Claude Fable 5.1), one session per task. Variance across sessions and across models is unmeasured.
- The SST-2 revision used the dev set, and the two MNIST revisions each inspected 500 training labels; the "labels seen" columns are the honest accounting, but the comparison is not perfectly matched because a human-in-the-loop revision and a training run use labels differently.

- The trained baselines are deliberately simple (bag-of-words, 1-hidden-layer MLP). Pretrained models reach 95%+ on SST-2 and 99%+ on MNIST; the claim is about the low-label regime only.
- The text-quality evaluation is a proxy (FineWeb-2's own filter decisions), not human quality judgements, and no trained baseline on the same proxy is reported here.
- The parameter counts of trained baselines and hand-written networks are not directly comparable: a trained logistic regression has a weight for every bigram it saw; a hand-written one has a weight for every word the author chose to think about.

9. Reproducibility

Every number in this paper is produced by a script in the repository, and the git history contains the blind version, each revision, and the baselines. Data are included (SST-2 TSVs, MNIST IDX files) except FineWeb-2, which the text-quality evaluation script streams from Hugging Face. The complete Claude.ai transcripts in which the weights were written are available from the first author.

References

- J. Lauritzen. "Got fable to hand-write weights of a 100k param model to recognize real versus fake legal citations." X post, 3 Nov 2026.
- R. Socher et al. "Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank." EMNLP 2013.
- Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. "Gradient-based learning applied to document recognition." Proc. IEEE, 1998.
- B. Moëll. OpenEuroLLM quality gate, AI Sweden, 2026 (internal repository; the hand-written network is reproduced in this repository under `text_quality/`).
- A. Wang et al. "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding." ICLR 2019.
- G. Penedo et al. "FineWeb2: One Pipeline to Scale Them All." 2024.
- J. Rae et al. "Scaling Language Models: Methods, Analysis & Insights from Training Gopher." 2021.